

Triadic Runtime — Design Record

Companion to the Triadic Runtime Specification v1.0-draft

This document is **informative, not normative**. Nothing here constrains a conforming implementation — that is the job of the specification alone. This document exists to preserve *why* each rule in the specification exists: what was tried, what broke, and what that breakage forced.

The methodology followed throughout is stated here explicitly because it produced every result in this record:

The Principle of Minimal Elevation. A concept is promoted into the ontology only after repeated attempts to derive it from existing primitives have failed. When a failure occurs, the required addition is introduced at the lowest possible layer — well-formedness before axioms, axioms before ontology — never higher than the failure demands.

Every chapter below ends with a **Result** block in the same format, so the pattern is visible across the whole record: how often something that looked like it might force a new primitive instead resolved as a smaller, lower-layer fix.

Chapter 1 — Why a Single Primitive Wasn't Enough

The starting question was not "what software should exist" but a narrower one: is there a smallest primitive from which every intelligent or coordinating system's behavior can be derived?

The first candidate was **prediction → observation → update** — the loop underlying control systems, reinforcement learning, predictive-processing accounts of cognition, and the scientific method. It's a real and useful primitive, but attempting to force databases, Git, and financial ledgers into this frame failed cleanly: none of them are trying to approximate an independent reality. A ledger balance isn't a prediction checked against ground truth — it's authoritative *by construction*. Forcing the frame onto these systems produced explanations that were technically not-false but explained nothing an engineer of those systems would recognize as the hard part of their job — the sign of a failed generalization rather than a successful one.

This produced the first real distinction: **epistemic systems** (tracking something external and independently real) versus **constitutive systems** (authoring and preserving institutional facts by declaring them). Prediction-observation-update governs the first. It has nothing to say about the second.

Result:

- New primitive? No — refined into two distinct primitives instead of one.
- New axiom? N/A (pre-ontology).
- Well-formedness rule? N/A.

- Theorem? None yet — this chapter set up the vocabulary (epistemic vs. constitutive) that later became Observation and Commitment.
-

Chapter 2 — Direction of Fit, and the Third Primitive

Formalizing "epistemic vs. constitutive" led to Searle's direction-of-fit distinction: word-to-world (a belief must match reality; reality is never wrong) versus world-to-word (the declaration constitutes the fact; there is no independent reality to check against).

This gave two primitives — Observation and Commitment — but a third case resisted both: a goal aimed at a *future* state that doesn't yet exist. Unlike an observation, failure to achieve a goal doesn't falsify it. Unlike a commitment, failure doesn't get silently absorbed as "that's just how it is now" — the gap stays open and demands more action. This became **Intention**, with its own direction of fit (world $\leftarrow$ effort: the action updates on mismatch, not the goal, and not the world by fiat).

Result:

- New primitive? Yes — Intention, forced by a genuine third case neither Observation nor Commitment could represent.
 - New axiom? N/A (pre-ontology).
 - Well-formedness rule? N/A.
 - Theorem? None yet.
-

Chapter 3 — The Raft Counterexample

Before treating the ontology as settled, an explicit test was run: can every meaningful software system be expressed as this loop without losing essential information? Git, CRDTs, and double-entry bookkeeping derived cleanly.

Raft did not. Raft's correctness guarantee depends on contacting a majority of nodes before any write is considered committed — a minority-partitioned node cannot make progress, by design. This is a direct, structural contradiction with what became Local Sufficiency (a node must be able to act using only local state). No amendment could resolve this, because it isn't a gap in the theory — it's CAP theorem surfacing inside the ontology. The runtime is AP-family by construction; Raft and other CP-family systems are not implementations of it and were never going to be.

This produced the first Boundary Theorem in embryonic form, well before Section 9 existed as a section: knowing exactly where a theory's boundary is, and stating it rather than hiding it, is stronger evidence of rigor than pretending the theory has none.

Result:

- New primitive? No.
- New axiom? No — confirmed Local Sufficiency as correctly scoped rather than over-broad.
- Well-formedness rule? No.

- Theorem? The AP boundary (later Section 9.1, Availability Boundary).
-

Chapter 4 — Identity: The First Adversarial Construction

Identity was chosen as the first hard test because it combines observation, trust, delegation, conflict, offline operation, and uncertainty with no central authority — a genuine stress test of the whole ontology at once, not just one primitive.

The construction: a self-signed genesis Commitment binds a key to a subject; later Commitments causally reference and supersede it for rotation; third-party belief accumulates as Observations, queried as a decayed confidence score, never stored as a fact.

Two real gaps surfaced:

1. **Genesis records have no causal predecessor.** The Causality invariant as originally stated ("every record references what justifies its existence") had no base case. Fixed by defining Genesis Record as a first-class term with its own validity mechanism (self-authorization), not a special exception bolted onto Causality.
2. **Authorization needed two forms.** Delegated authorization (a chain back to a prior commitment) and self-authorization (cryptographic proof, no chain) are structurally different, and the original wording only accounted for the first.

A third result was a genuine impossibility, not a gap: if someone loses every key with no causal link to the old identity, no path back exists. No amount of vouching converts an Observation into a Commitment — this became Bounded Continuity (8.2.5), one of the cleanest Boundary Theorems in the final specification, because it's a *consequence*, not a design choice.

Result:

- New primitive? No.
 - New axiom? No — two refinements (genesis base case, dual authorization) to Causality and the authorization concept.
 - Well-formedness rule? Yes — became 6.5 (Genesis Records).
 - Theorem? Bounded Continuity (8.2.5).
-

Chapter 5 — Semantics: Meaning Without a Fourth Layer

The concern: a runtime this general would eventually need to know what "household" or "maize" *means* — seemingly requiring a Domain Semantics layer between Policy and the Runtime.

Attempted construction instead: a Schema is just a Commitment payload defining a vocabulary, like any other institutional fact. Two communities with incompatible definitions produce no runtime error — Non-Silent Conflict handles semantic mismatch the same way it handles double-spend: both are preserved, neither silently coerced. Comparison across schemas without an explicit mapping Commitment is simply undefined, not resolved by guessing.

This eliminated an entire proposed architectural layer. Semantics is not infrastructure — the runtime only ever needs to answer "which definition does this record claim to use," never "what does this term mean." A later attempt to make Schema Mapping its own well-formedness rule (6.6) was rejected on the same grounds: a mapping is an ordinary Commitment referencing two schemas, nothing more, per Minimal Elevation.

Result:

- New primitive? No.
 - New axiom? No.
 - Well-formedness rule? Yes — 6.4 (Schema Reference), later extended in-place to cover mapping rather than given a separate rule.
 - Theorem? Local Meaning (8.2.4), later recognized as a specialization of the more general Query-State Separation (8.2.1) discovered in Chapter 9.
-

Chapter 6 — Workflow: Stressing the Intention Lifecycle

Workflow was chosen to stress hierarchy, retry, timeout, compensation, and concurrency — none of which had been tested against a single Intention in isolation.

Findings:

- **Composition:** child intentions causally reference parent intentions — no new edge type, but the first confirmation that Causality's "record references predecessor" applies between intentions, not only from intention to observation/commitment.
- **Closure across hierarchy:** the runtime cannot enforce "parent closes only after children close" without violating Local Sufficiency (synchronous dependency tracking). This had to be stated as an explicit limit rather than assumed: Closure Is Strictly Local (8.1.5).
- **Retry:** a failed intention cannot reopen — Immutability's spirit extends to terminal states even though intentions aren't literally immutable payloads. A retry is always a new intention referencing the failed one. This produced the Intention lifecycle rule (`(active → {completed, cancelled, expired})`, no reopening) now stated directly in Section 3.4.
- **Timeout:** if nothing witnesses an expiry, who authors the required Closure observation? This forced `(origin: witnessed | inferred)` (6.3) — the first case where the runtime's own clock, not a person or sensor, is the legitimate author of an observation.
- **Compensation:** rollback is impossible under Immutability; a correction is always a new counter-commitment.
- **Concurrent workflows on one resource:** confirmed Non-Silent Conflict applies to concurrent Intentions, not only Commitments — over-allocation becomes visible only after the fact, never prevented.

Result:

- New primitive? No.
 - New axiom? No.
 - Well-formedness rule? Yes — 6.3 (Observation Origin), forced specifically by the timeout case.
 - Theorem? Acyclicity (8.1.3) and Closure Is Strictly Local (8.1.5).
-

Chapter 7 — Contracts: Mutual Commitment Without a Joint-Authorization Primitive

The test: can two independent, mutually distrusting authorities create one shared, binding fact without a new "agreement" primitive?

Construction: an offer is a self-authorized Commitment; an acceptance is a second party's Commitment causally referencing the offer's terms-hash exactly. "The contract" is never stored — it's a query pattern (offer + matching acceptance) evaluated at read time. Breach is an Observation; termination is a further Commitment, valid or not according to policy the runtime never adjudicates.

One real implementation fork was surfaced, not resolved: offer/acceptance as two independent commitments allows a permanently dangling, unaccepted offer; true multisig (one record requiring both signatures atomically) closes that gap but is a materially different mechanism. Both are valid under this specification; neither is mandated.

A sharper version of Local Meaning also emerged here: a contract's *validity* can change even when no record in the graph changes, if the interpreting policy itself changes later (e.g., a jurisdiction rules a clause unenforceable). This showed that Query-State Separation is more general than "records need a schema" — it's that *nothing computed from the graph is ever fixed*, because computation itself is never a record.

Result:

- New primitive? No.
 - New axiom? No.
 - Well-formedness rule? No new rule — reused 6.1 and 6.2.
 - Theorem? Corollary to Local Meaning (8.2.4): interpretation can change independent of the record.
-

Chapter 8 — Multi-Agent Coordination: No Global Planner

The hardest test proposed before Capability Security: can multiple independent, mutually unaware authorities coordinate — negotiate, delegate, conflict, recover from a crashed peer — without a "coordination" object or a global planner?

Every mechanism reduced: shared observations are consumed independently by each agent with no synchronization (Local Sufficiency); disagreement between agents is two unresolved intentions (Non-Silent Conflict); negotiation is repeated offer/counter-offer, i.e.,

repeated Chapter 7; delegation is ordinary authorization; a crashed-and-recovered agent simply receives new graph branches on reconnect, no different from Git.

Planning itself dissolved. No agent ever held the global plan — it was reconstructible only by replaying every agent's independent records after the fact. This produced Emergent Coordination (8.2.6): coordination is not stored, it is a query over history under a chosen policy.

One near-miss: an early attempt introduced "Mediated Influence" as if it were an independent rule (intentions can't cite other authorities' intentions directly). This was initially miscategorized as a theorem, but later, correctly, recognized as not a structural consequence of the axioms at all — it only became necessary once Capability Security (Chapter 10) exposed *why* it mattered. It now lives as an informative consequence inside 6.1, not a numbered theorem, because giving it equal weight with genuinely axiom-derived results like Acyclicity would have overstated what was actually discovered at this stage.

Result:

- New primitive? No.
 - New axiom? No.
 - Well-formedness rule? No new rule at this stage (Mediated Influence's real cause wasn't identified until Chapter 10).
 - Theorem? Emergent Coordination (8.2.6), Policy Neutrality (8.2.3, formalizing a Design Principle that had been informal until this point).
-

Chapter 9 — Query-State Separation, Named

Reviewing Chapters 4–8 together surfaced a pattern that had been operating unnamed the whole time: Reputation, Identity confidence, Contract validity, and now Coordination itself were all the same move — a persistent record, plus a function computed over it at read time, with the computed value never itself stored.

This was promoted to its own theorem, **Query-State Separation** (8.2.1), and the earlier results were re-derived as corollaries rather than independent findings: Transient Interpretation, Policy Neutrality, and Local Meaning all follow from it directly. This tightened the theorem section's dependency structure considerably — what had been a flat list of eight or nine unrelated-looking results turned out to mostly be one result applied in different contexts.

Result:

- New primitive? No.
 - New axiom? No.
 - Well-formedness rule? No.
 - Theorem? Query-State Separation (8.2.1), the root of the Emergent branch of Section 8.
-

Chapter 10 — Capability Security: The First Real Gap

This is the chapter that produced a genuine finding, not just another clean derivation — worth reading in full because it's the strongest evidence in this whole record that the process was a real attempt to break the theory, not a confirmation exercise.

Attack 1 (Intention Forgery). Alice creates an intention to distribute medicine. Bob creates an intention to unlock a warehouse, causally citing Alice's intention as its cause. Checked mechanically against every invariant as they stood at the time — Immutability, Causality, Local Sufficiency, Closure, Non-Silent Conflict — the graph was valid. Nothing stopped Bob. The exploit: causality was being silently treated as authorization. Nothing in the ontology ever justified that inference; it had simply never been tested.

Attack 2 (Delegation Forgery) and **Attack 3 (Observation Laundering)** confirmed the same root cause from different angles: an intention citing a legitimate observation or another legitimate intention as its cause was, at the time, sufficient for the citing intention to appear well-formed, regardless of whether its author had any actual authority to act.

Diagnosis. This was not a missing mechanism — every earlier chapter's use of Commitments had already required authorization from the start (it was in the original invariant list, first drafted before this whole derivation program began). The bug was an asymmetry: **Intentions had quietly never been held to the same standard.** Every prior "successful" derivation involving intentions had happened not to expose this, purely by the shape of the test case, not because the ontology handled it.

Attempted fix and further attack. Authorization-by-graph-traversal (checking a chain of delegation Commitments back to a genesis or valid grant) defeated four further attacks cleanly — copied delegation hashes, fake delegations, expired delegations, revoked delegations, all correctly rejected by traversal. But **Attack 5 (Authority Amplification)** found a real remaining hole: Alice delegates "allocate up to 100kg" to Bob; Bob re-delegates full authority to Eve. Nothing in the original delegation payload said whether delegation was itself delegable. This was the one case that didn't resolve from existing rules and genuinely required a new addition.

The fix, and its correct placement. Not a new primitive, not a new axiom — a well-formedness rule, generalized beyond delegation specifically to any capability grant: transitivity must be explicit, defaulting to non-transitive. And a symmetry restoration, not a new mechanism: Intentions must satisfy the same authorization-traceability requirement Commitments always had.

Result:

- New primitive? No.
- New axiom? No.
- Well-formedness rule? Yes, two: Authorization Traceability extended explicitly to Intentions (6.1), and Capability Transitivity generalized beyond delegation (6.2).
- Theorem? Authorization Traversal Fails Safe (8.1.4) — absence of a valid path is always denial, never implicit grant. Also produced the Mediated Influence

consequence, finally correctly placed inside 6.1 rather than as an independent theorem, since its necessity only became clear once this chapter identified the actual mechanism (authorization forgery via bare causal reference) it prevents.

Chapter 11 — Freezing the Kernel

After ten chapters of adversarial construction across identity, semantics, workflow, contracts, multi-agent coordination, and capability security, the pattern had stabilized: every difficult case resolved as one of four things — an axiom refinement, a derived theorem, a well-formedness rule, or a boundary theorem — and never as a fourth persistent primitive. This consistency, under repeated and increasingly targeted attack, was treated as the signal to stop expanding the ontology and begin specifying it formally.

The specification (Sections 1–9) was drafted directly from this record, with one discipline maintained throughout: each invariant in Section 4 carries exactly one normative requirement, with exceptions and special cases pushed down to Section 6 rather than folded into the invariant itself. Two corrections were caught during drafting that hadn't been caught during derivation:

- Causality, as originally phrased, mixed structural dependency with authorization — precisely the conflation Chapter 10 later showed to be exploitable. Drafting forced this apart into a purely structural Causality (4.2) and a separate Authorization Traceability rule (6.1).
- The phrase "authorization before execution" (an early attempt to state the Chapter 10 fix) was caught as a category error during drafting: the runtime has jurisdiction over record creation, never over what happens in the world afterward. This became 9.5 (No Enforcement).

Result:

- New primitive? No — final count after eleven chapters: zero.
 - New axiom? No — final count: five (Immutability, Causality, Local Sufficiency, Closure, Non-Silent Conflict), each stated with a single normative requirement.
 - Well-formedness rules discovered across the whole process: five (Authorization Traceability, Capability Transitivity, Observation Origin, Schema Reference, Genesis Records).
 - Theorems discovered: twelve, split five structural / seven emergent, with Query-State Separation identified as the root of the emergent branch.
 - Boundary theorems stated: six, each framed as a non-goal rather than a limitation.
-

Appendix — Methodology Summary

Across all eleven chapters, every attempted "hard case" was handled by exactly one of four outcomes, and never a fifth:

Outcome	What it means	Count
Axiom refinement	An existing invariant's wording was incomplete, not wrong	2 (Causality's genesis base case; Causality's structural/authorization split)
Derived theorem	True as a consequence, without needing to be asserted	12
Well-formedness rule	A validity constraint on a payload or record type, below axiom level	5
Boundary theorem	An explicit non-goal, stating what the runtime does not attempt	6

No case in this record produced a fourth persistent primitive beyond Observation, Commitment, and Intention. This is not proof of completeness — no adversarial process can prove a negative for all future cases — but it is the evidentiary basis on which Sections 1–9 were frozen as v1.0-draft rather than left as an open research question.